05. Cloud Deployment & Serverless Operations

This document describes the cloud-native, serverless execution framework developed for **PredSea** using **Google Cloud Platform (GCP) Batch**, **Spot VM instances**, and automated cost-protection mechanisms.

---

1. Containerized HPC Engine (`predsea-croco-staging`)

Rather than maintaining dedicated, static HPC server hardware, the entire numerical modeling environment—including compiled Fortran 90 binaries for CROCO 2.1.3 and SWAN 41.45, OpenMPI execution runtimes, NetCDF C/Fortran libraries, and Python spatial post-processors—is encapsulated in an immutable Docker container image.

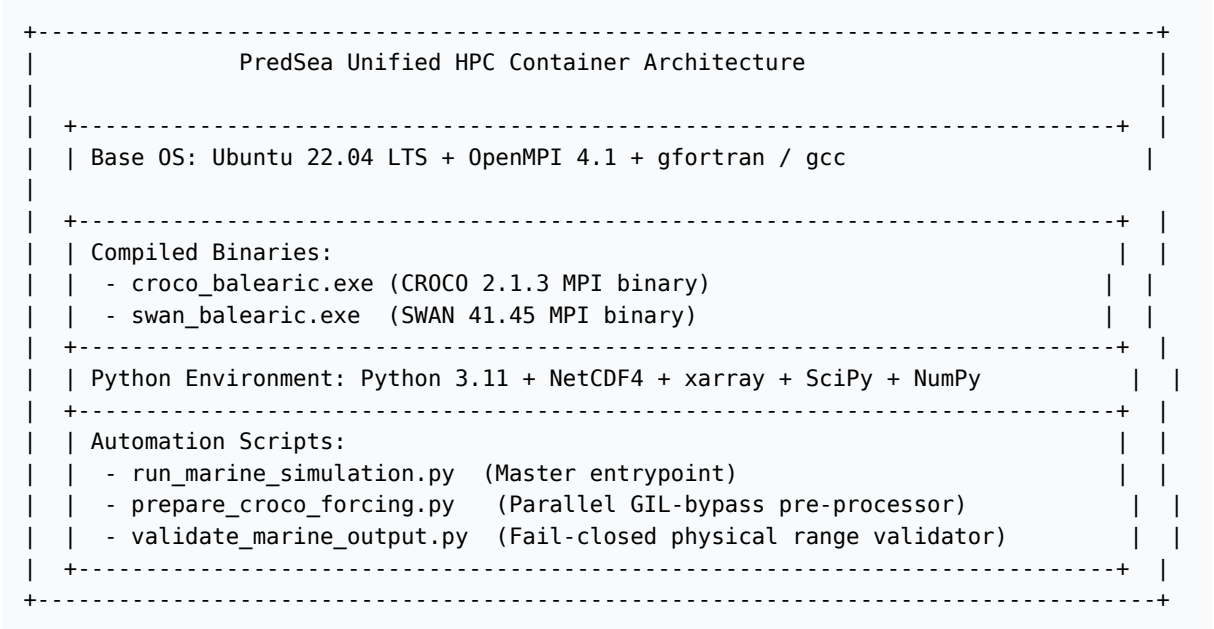

Image digests are pinned in GCP Artifact Registry: `europe-west1-docker.pkg.dev/predsea-api/predsea-croco-staging/croco:latest`

---

2. Dynamic Resource-Aware Compute Sizing

To prevent Out-Of-Memory (OOM) failures while minimizing compute expenditure, the submission orchestrator `scripts/submit_gcp_batch_simulation.py` dynamically parses regional geographic bounds and computes total spatial grid points:

$$\text{Grid Points} = \left(\frac{(\text{Lat}_{\max} - \text{Lat}_{\min}) \times 111,000}{H_{\text{res}}} \right) \times \left(\frac{(\text{Lon}_{\max} - \text{Lon}_{\min}) \times 111,000 \times \cos(\text{Lat}_{\text{mid}})}{H_{\text{res}}} \right)$$

Based on the calculated density, the job launcher selects the optimal Compute Engine Spot VM template and MPI rank layout:

```
                [Geographic Region Config JSON]
                              |
                 (Compute Sizing Formula)
                              |
        +---------------------+---------------------+
        | < 500k Points       | 500k - 2M Points    | > 2M Points
        v                     v                     v
  [c2d-highcpu-4]       [c2d-highcpu-8]       [c2d-highcpu-16]
  (2 MPI Ranks)         (4 MPI Ranks)         (16 MPI Ranks)
```

Region Provisioning Matrix

Region ID	Grid Points	Machine Type	Spot Hourly Rate	MPI Ranks	Compute Cost / 24h Forecast
Balearic 1km	$200,901$	`c2d-highcpu-16`	**$0.180 / hr**	16	**$0.063** (21 min run)
Alboran 1km	$125,751$	`c2d-highcpu-4`	**$0.045 / hr**	2	**$0.015** (20 min run)
Gulf of Lion 1km	$135,951$	`c2d-highcpu-4`	**$0.045 / hr**	2	**$0.015** (20 min run)
Tyrrhenian 1km	$423,801$	`c2d-highcpu-8`	**$0.090 / hr**	4	**$0.036** (24 min run)
Algerian 1km	$286,251$	`c2d-highcpu-8`	**$0.090 / hr**	4	**$0.030** (20 min run)
Total Med Suite	**~1.17 Million**	**5x Spot VMs**	–	**28 Ranks**	**$0.172 / daily run**

---

3. Spot VM Preemptibility & Cost Safety Traps

Compute costs are reduced by **70%–90%** by leveraging GCP **Spot Instances**. To ensure fault tolerance and prevent runaway cloud billing:

```
flowchart TD
    Submit["submit_gcp_batch_simulation.py"] --> Launch["GCP Batch Provisions Spot
VM"]
    Launch --> TrapSet["Set Bash Exit Traps & Timers"]

    TrapSet --> RunSim["Execute CROCO MPI Run"]

    RunSim -- "SUCCESS" --> Valid["validate_marine_output.py"]
    RunSim -- "SPOT Preemption / Crash" --> FailureHandler["Capture Failure
Diagnostics"]

    Valid -- "PASS" --> Upload["Sync NetCDF to GCS Staging"]
    Valid -- "FAIL (Physical Range)" --> FailureHandler

    Upload --> WriteSuccessMarker["Write SUCCESS Token"]
    FailureHandler --> WriteFailMarker["Write FAILURE Marker"]

    WriteSuccessMarker --> SelfDestruct["Auto Self-Deletion Trap"]
    WriteFailMarker --> SelfDestruct
```

1. **Automated Self-Deletion Traps (`vm_startup.sh`)**: The container launcher registers a shell `trap` command (`trap 'gcloud compute instances delete ... --quiet' EXIT INT TERM`). Regardless of whether the simulation finishes successfully or encounters a script crash, the compute node automatically self-destructs, eliminating idle billing risks.
2. **Run-Scoped Immutability**: Outputs are saved under immutable run paths: `gs://predsea-daily-outputs-test/predictions/YYYY-MM-DD/runs/[RUN_ID]/`
3. **Fail-Closed Diagnostics**: If a Spot node is preempted by GCP, the master orchestrator detects missing output hashes, marks the run as `FAILED`, preserves intermediate log files, and triggers a clean retry with a new unique run ID.

---

4. Output Consolidation & Cloud Storage Pipeline

Once numerical solver integration completes, raw model outputs undergo post-processing:

1. **Canonicalization**: Intermediate output files are consolidated into standard single-file NetCDF4 products (`balearic_1km_croco_forecast.nc`).
2. **ETL BigQuery Ingestion**: `scripts/ingest_predictions_to_bigquery.py` extracts time series across **3,046 canonical harbors** and **127 shipping routes**, inserting records into BigQuery tables for sub-millisecond API lookups.
3. **FastAPI Cloud Run Serving**: The web API fetches predictions from BigQuery and GCS, generating real-time Vector Tile overlays for Deck.gl map visualization.